KALASALINGAM UNIVERSITY

PROJECT REPORT

ONLINE COURSE MARKETPLACE

A web-based platform to discover, purchase and learn from online courses

Submitted by

G.V.Vara Prasad — 99230040025

K. Gangaiah — 99230040032

N. Sri Charan — 99230040043

T. Koushik — 99230040062

B. Vamsi — 99230040013

B. Benarji Kumar Reddy — 99230040011

Submitted to
Kalasalingam University

Academic Year: 2026-2027

Project Guide: Ajusha
Department: Cse

S. No.	Student Name	Register Number
1	G.V. Vara Prasad	99230040025
2	K. Gangaiah	99230040032
3	N. Sri Charan	99230040043
4	T. Koushik	99230040062
5	B. Vamsi	99230040013
6	B. Benarji Kumar Reddy	99230040011

ABSTRACT

The Online Course Marketplace is a web-based application designed to bring course discovery, enrolment and learning into one place. A learner can browse available courses, search and filter the catalogue, view course information, add courses to a cart or wishlist, and proceed through a checkout flow. After enrolment, the learner can open the learning area, move through lessons and keep track of completed content.

The application is developed with a React front end and a Node.js/Express back end. Supabase is used for authentication and data access. The server exposes API routes for courses, categories, reviews, cart operations, enrolments, instructor functions and administration. The payment flow is integrated with Razorpay, while an optional question-and-answer feature uses Google Gemini through a server-side service.

The main purpose of the project is to demonstrate how a role-oriented online learning marketplace can be organised as a full-stack application. The report explains the problem, objectives, architecture, modules, technologies, implementation approach and test cases. Actual deployment details and live test results should be filled in based on the environment in which the team demonstrates the application.

TABLE OF CONTENTS

1. Introduction

2. Problem Statement and Objectives

3. Existing and Proposed System

4. Requirements and Technology Stack

5. System Design and Architecture

6. Module Description

7. Database and Data Handling

8. Implementation Details

9. Testing and Validation

10. Results and Discussion

11. Limitations and Future Enhancements

12. Conclusion

References

Appendix A – Setup Notes

1. INTRODUCTION

1.1 Background

Online learning has become a common way for students and working professionals to develop new skills. Courses are available in areas such as programming, design, data science and business. However, learners may have to visit different websites to compare courses, manage purchases and access learning material. A single platform can make this process easier by keeping the main activities together.

1.2 About the Project

The Online Course Marketplace is intended to support three main activities: discovering courses, purchasing or enrolling in courses, and learning from the enrolled content. The application also includes separate areas for students, instructors and administrators. This structure helps represent the different responsibilities involved in running a course platform.

1.3 Scope

- Display a catalogue of courses with search, category and sorting options.
- Show course details and learner reviews.
- Support user registration, login and password recovery through Supabase Authentication.
- Provide cart and wishlist features for learners.
- Support checkout using a Razorpay order and payment verification flow.
- Allow enrolled learners to access course lessons and record lesson completion.
- Provide dashboard routes for student, instructor and administrator roles.
- Expose an AI question endpoint backed by Google Gemini.

1.4 Intended Users

User	Main activities
Student / Learner	Browse courses, maintain a wishlist or cart, enrol, study lessons and track progress.
Instructor	Use the instructor dashboard to access instructor-related functions.
Administrator	Access the admin dashboard for platform-level administration.
Visitor	View public pages, explore courses and create an account.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Learners need a convenient way to find relevant courses and continue learning after enrolment. When course information, purchase steps and learning progress are handled separately, the user experience can become confusing. The project addresses this by building a common web application for course browsing, cart management, payment processing and lesson access.

2.2 Objectives

- Build a responsive and easy-to-navigate course marketplace interface.
- Organise courses so that users can search, filter and sort the available options.
- Provide account registration and login for personalised activities.
- Connect the front end to server-side APIs for course, cart, review and enrolment operations.
- Provide a checkout process with server-side price calculation and payment-signature verification.
- Give enrolled learners a structured lesson view with completion tracking.
- Separate student, instructor and administrator areas according to user roles.
- Explore the use of an AI service to answer learner questions.

2.3 Expected Outcome

The expected outcome is a working prototype in which a learner can move from course discovery to checkout and then to the learning page. The project also demonstrates a modular full-stack structure that can be extended with additional course-management and reporting features.

3. EXISTING AND PROPOSED SYSTEM

3.1 Existing Approach

In a basic online learning workflow, learners may discover courses on one site, keep notes about courses elsewhere, and manage learning progress manually. Smaller course portals may also provide limited filtering or have separate experiences for learners and course providers. These differences can make it harder to manage the complete learning journey.

3.2 Proposed System

The proposed application combines the major stages of the learner journey. The home and courses pages provide discovery; course detail pages provide more information; cart and checkout pages support purchasing; and the learning page provides access to the curriculum. Account and dashboard pages support personalised and role-based activities.

3.3 Advantages

- Course discovery and learning are available through one application.
- Search, category filters and sorting reduce the effort required to find a course.
- Cart and wishlist features let users organise courses before purchase.
- The payment process checks course availability and calculates prices on the server.
- Lesson completion can be recorded and used to show learning progress.
- The modular API structure makes future maintenance and expansion easier.

4. REQUIREMENTS AND TECHNOLOGY STACK

4.1 Hardware Requirements

Component	Minimum practical requirement

Processor	Modern dual-core processor or better
RAM	4 GB minimum; 8 GB recommended for development
Storage	At least 1 GB free space for source, dependencies and project files
Display	1366 × 768 resolution or higher recommended
Network	Internet connection for hosted services and package installation

4.2 Software Requirements

Software / Service	Purpose
Windows, Linux or macOS	Development environment
Node.js and npm	Run the front end and back end and manage packages
React 19	Build the component-based user interface
Vite	Front-end development server and build tool
Node.js with Express	Implement server-side APIs
Supabase,MongoDB	Authentication and database access
Razorpay	Payment order creation and verification
Google Gemini API	AI-powered question answering
Web browser	Access and test the application

4.3 Technologies Used

Frontend: React, React Router, Axios, React Hook Form and Lucide React icons. Vite is used as the development and build tool. The source includes context providers for authentication, cart and wishlist state.

Backend: Node.js and Express provide the API layer. Express middleware handles JSON requests, CORS, request logging, validation, authentication checks and centralised error responses.

Services: Supabase client libraries are used for authentication and database operations. Razorpay is used in the payment routes. The Google GenAI SDK connects the AI question endpoint to the Gemini 2.5 Flash model.

4.4 Development Environment

The project is separated into client and server folders. The client uses ES modules and Vite scripts. The server uses CommonJS modules and provides start and development scripts. Configuration values are supplied through environment files; secret values must remain on the server and should not be committed to source control.

5. SYSTEM DESIGN AND ARCHITECTURE

5.1 Architecture Overview

The application follows a client-server architecture. The browser displays the React application. When data is required, the client sends HTTP requests through Axios to the Express API. The server validates

requests, checks authentication where required and communicates with Supabase or external services. The response is then used by the React pages to update the interface.

Layer	Main responsibility	Project elements
Presentation layer	Screens, navigation and user interaction	React pages, layouts, components and contexts
API layer	Receive requests and return responses	Express app, route files and controllers
Business/service layer	Perform application operations	Course, cart, enrolment, authentication and AI services
Data and external services	Persist data and handle hosted functions	Supabase, Razorpay and Google Gemini

5.2 Request Flow

1. A user opens a page or performs an action in the browser.
2. The React component calls a service function, usually through the shared Axios client.
3. The client attaches the available Supabase access token and a session identifier to API requests.
4. Express routes direct the request to the appropriate controller or route handler.
5. The server checks input and authentication, then accesses Supabase or the relevant external service.
6. The API returns a response; the front end displays the result or an error message.

5.3 Main Application Routes

Route	Purpose
/	Home page
/courses	Course catalogue and filters
/courses/:id	Course details
/login, /register	Account access
/cart, /wishlist	Saved and selected courses
/checkout	Checkout for authenticated users
/my-courses	Learner's enrolled courses
/learn/:courseId	Course learning interface
/dashboard	Student dashboard
/instructor	Instructor dashboard
/admin	Administrator dashboard
/about, /contact	Informational pages

5.4 Security Design

The front end includes protected routes to redirect unauthenticated users to the login page and to check the role required by selected dashboard routes. The server also has authentication middleware that validates bearer tokens with Supabase. Payment endpoints require authentication. The payment route calculates the order amount using course prices read from the database rather than trusting a total sent by the browser, and includes payment-signature verification.

The front-end route guard is a navigation aid, not a security boundary. Sensitive operations must always be checked on the server. Production deployment should also review database row-level security policies, role assignment, input validation, rate limiting, HTTPS and secret management.

6. MODULE DESCRIPTION

6.1 User Authentication

Registration, login, logout, current-user retrieval and password reset are implemented through Supabase Authentication. User metadata includes a full name and role. The client stores authentication state through an AuthContext provider.

6.2 Course Catalogue

The catalogue retrieves courses from the Express API. It supports text search, category selection, level and rating filters, sorting choices such as newest, popularity, rating and price, and paginated results. Featured, trending and top-rated catalogue requests are also supported.

6.3 Course Details and Reviews

A course detail page presents information about a selected course. Review API routes allow reviews to be retrieved for a course and submitted by users. A reusable star-rating component is present in the client.

6.4 Cart Management

The cart module lets a learner add courses, view selected items and remove items. Server-side cart service functions interact with the cart_items and courses data. A coupon endpoint and checkout handler are also defined in the cart routes.

6.5 Wishlist

The WishlistContext and wishlist page provide a place for users to keep courses they may want to study later. This avoids having to search for the same course again.

6.6 Payment and Checkout

The payment routes create a Razorpay order after checking the logged-in user and their cart. The server reads the selected course records, checks that the courses are published, calculates the total in Indian rupees and converts it to paise for Razorpay. A separate verification route checks the payment signature.

6.7 Enrolment and Learning

Enrolment routes provide access to a learner's enrolments and support lesson-completion and progress updates. The learning page loads the course curriculum, displays lessons, tracks completed lesson identifiers and calculates a completion percentage.

6.8 Student Dashboard

The student dashboard is available through a protected route requiring the student role. It provides a dedicated area for learner-related information and activities.

6.9 Instructor Dashboard

The instructor dashboard is available through a protected route requiring the instructor role. Instructor-related server functionality is grouped under the instructor API route.

6.10 Administrator Dashboard

The admin dashboard is protected by an admin role check. The server exposes an administrative dashboard endpoint for platform-level information.

6.11 AI Question Assistant

The server exposes POST /api/ai/ask. It accepts a question, passes it to the Gemini service and returns the generated answer. The API key is read from the server environment rather than being placed in the browser code.

7. DATABASE AND DATA HANDLING

7.1 Data Storage

The source code connects to Supabase using the Supabase JavaScript client. The inspected server services refer to the courses, cart_items and enrollments tables. Authentication is handled by Supabase Auth. The project archive does not include a complete database schema or migration file, so the exact column definitions, relationships and constraints should be confirmed from the team's Supabase project before final submission.

7.2 Main Data Areas

Data area	Purpose in the application	Evidence in source
Courses	Course details, price, publication status and catalogue information	Course services and payment route query the courses table
Cart items	Associates a user's selected courses with their cart	Cart service and payment route use cart_items
Enrollments	Stores a learner's course enrolment and progress information	Enrolment service and learning page
Authentication users	User identity and session information	Supabase Auth sign-up, sign-in and get-user calls
Reviews	Learner feedback associated with courses	Review routes and controller

7.3 Data Flow and Integrity

The front end requests data through service modules rather than directly embedding database operations in every page. The server-side services provide a common place for database queries. In the payment process, the server retrieves the current course prices and checks publication status before creating an order. This is important because the browser should not be treated as a trusted source for the final payable amount.

7.4 Database Diagram Note

For the final institutional copy, insert the actual Supabase table relationship diagram here. It should be exported from the database schema used in the demonstration. Do not create relationships or field names based only on assumptions.

8. IMPLEMENTATION DETAILS

8.1 Front-End Structure

The client source is organised into pages, reusable components, layouts, contexts, hooks, services and utility files. App.jsx defines the route structure and wraps the application in providers for authentication, cart and wishlist state. A React error boundary provides a fallback screen if a component throws an error.

8.2 API Structure

The Express application configures CORS, JSON parsing, URL-encoded request parsing and development request logging. It registers route groups for authentication, courses, categories, reviews, cart, enrolments, instructor and admin operations, and payments. A health-check endpoint is available at /api/health. Unknown routes return a 404 response and a central error handler processes application errors.

8.3 API Endpoint Summary

API group	Selected operations
Authentication	POST /api/auth/register, /login, /forgot-password
Courses	GET /api/courses, GET /api/courses/:id, POST /api/courses, PUT and DELETE by ID
Categories	GET /api/categories and GET /api/categories/:id
Reviews	GET /api/reviews/course/:courseId, POST /api/reviews
Cart	GET /api/cart, cart item operations, coupons and checkout
Enrolments	GET /api/enrollments, POST enrolment and progress operations
Instructor	GET /api/instructor/dashboard
Admin	GET /api/admin/dashboard
Payments	POST /api/payments/create-order and /verify
AI assistant	POST /api/ai/ask

8.4 Configuration

The client expects a VITE_API_URL value for the API base URL and Supabase client configuration. The server reads environment variables for its port, client origin, Supabase connection, Razorpay credentials and Gemini API key. The .env.example files should be used as the starting point. Real credentials must be entered locally and kept private.

8.5 Running the Application

7. Install a compatible Node.js LTS release and npm.
8. Open a terminal in the client folder and install dependencies using npm install.

9. Create the client environment file using the supplied example and enter the required Vite and Supabase settings.
10. Open another terminal in the server folder and install dependencies using npm install.
11. Create the server environment file using the supplied example and configure Supabase, Razorpay and Gemini values as needed.
12. Start the backend with npm run dev from the server folder.
13. Start the frontend with npm run dev from the client folder.
14. Open the local address printed by Vite in a browser. Confirm the backend health endpoint at http://localhost:5000/api/health.

Note: The actual environment variable names and required values should be checked against the two .env.example files. Never copy real API keys into the report or share them in screenshots.

9. TESTING AND VALIDATION

Testing is necessary to check that the main user journeys work correctly and that invalid requests are handled safely. The following table is a suggested test plan based on the modules and routes found in the source code. Mark each result as Pass or Fail only after carrying out the test in the team's configured environment.

Test ID	Test scenario	Expected result	Actual result
TC-01	Open the home page	Home page loads without a runtime error	To be filled
TC-02	Register with valid details	Account creation succeeds or confirmation guidance is shown	To be filled
TC-03	Log in with valid credentials	User session is established and protected pages become accessible	To be filled
TC-04	Open the course catalogue	Course list is displayed from the API	To be filled
TC-05	Search and filter courses	Results reflect the selected search and filters	To be filled
TC-06	Open a course detail page	Selected course information is displayed	To be filled
TC-07	Add/remove a course from cart	Cart contents update correctly	To be filled
TC-08	Attempt checkout while logged out	User is asked to log in	To be filled
TC-09	Create and verify a test payment	Order and signature verification follow the payment flow	To be filled
TC-10	Open an enrolled course	Curriculum and lesson view are displayed	To be filled
TC-11	Mark a lesson complete	Lesson completion and progress update	To be filled
TC-12	Open a role-protected dashboard with the wrong role	User is redirected or denied access	To be filled
TC-13	Send an empty AI question	API returns a validation response	To be filled
TC-14	Open an unknown URL	Not-found page or 404 response is shown	To be filled

9.1 Testing Notes

The project package contains a test script in the server package that currently reports that no automated tests are configured. Therefore, this report does not claim that the above test cases have already passed. The team should execute them, record the actual results and attach screenshots or logs from the demonstration.

9.2 Error Handling

The client Axios instance normalises API errors into readable messages. The application includes loading skeletons, error and empty-state components. On the server, an unknown route returns a 404 response and a central error handler is registered after the route definitions.

10. RESULTS AND DISCUSSION

The submitted source code provides the main structure of an online course marketplace. It includes separate client and server applications, user authentication, course browsing, cart and wishlist state, payment routes, enrolment and learning pages, role-based dashboard routes and an AI question endpoint.

The project demonstrates how several independent technologies can be connected in a single application. React manages the user interface, Express organises the APIs, Supabase provides authentication and data access, Razorpay handles the payment-order flow, and Gemini supplies generated answers for the AI assistant.

The final demonstration should capture the actual running application screens, including the home page, course listing, course detail, login/register, cart, checkout, student dashboard and learning page. Add screenshots from the working environment in the spaces below.

[Insert application screenshot here]
Figure 10.1 – Home page

[Insert application screenshot here]
Figure 10.2 – Course catalogue

[Insert application screenshot here]
Figure 10.3 – Course details

[Insert application screenshot here]
Figure 10.4 – Cart and checkout

[Insert application screenshot here]
Figure 10.5 – Learning page and progress

11. LIMITATIONS AND FUTURE ENHANCEMENTS

11.1 Current Limitations

- The archive does not contain a complete database schema or migration history, so the final table structure must be documented from the live Supabase project.
- The server package does not currently define an automated test suite.
- The availability and behaviour of external services depend on valid credentials, network access and provider configuration.
- The project uses role checks in the front end for route navigation; server-side authorisation should be reviewed for every privileged operation.
- Production readiness requires additional checks for database policies, rate limiting, monitoring, accessibility and deployment security.

11.2 Future Enhancements

- Add a full instructor course-creation workflow with lesson and media management.
- Add certificates after course completion.
- Provide richer learning analytics and downloadable progress reports.
- Add course recommendations based on learner interests and previous activity.
- Improve the AI assistant with course-specific context and safe response limits.
- Add automated unit, API and end-to-end tests.
- Improve accessibility, multilingual support and mobile usability.
- Add administrator tools for moderation, refunds, reports and platform analytics.

12. CONCLUSION

The Online Course Marketplace project brings course discovery, purchase and learning activities together in a single web application. It provides a React-based user interface, an Express API layer and integrations with Supabase, Razorpay and Google Gemini. The implemented modules cover course browsing, authentication, cart and wishlist features, payment processing, enrolment, lesson progress and role-specific dashboards.

Working on this project gives the team practical experience in full-stack development, API design, authentication, database access and integration with external services. The application can be improved further through more complete automated testing, stronger production security checks, richer instructor tools and additional learner-focused features. Overall, the project provides a useful foundation for an online learning marketplace.

REFERENCES

- React documentation – https://react.dev/
- Vite documentation – https://vite.dev/
- Express documentation – https://expressjs.com/
- Supabase documentation – https://supabase.com/docs
- Razorpay developer documentation – https://razorpay.com/docs/

- Google Gemini API documentation – https://ai.google.dev/gemini-api/docs
- Axios documentation – https://axios-http.com/
- React Router documentation – https://reactrouter.com/

Project source: prasadc1.zip, containing the client and server application source files supplied by the project team.

APPENDIX A – SETUP AND SUBMISSION CHECKLIST

- Fill in the department, programme, academic year and project guide details.
- Confirm the exact project title and spelling of all student names and register numbers.
- Add the university-prescribed certificate and declaration wording if a fixed format is required.
- Export the actual database schema or ER diagram from Supabase and insert it in Section 7.
- Run the application using the team's own environment variables and verify the main user flows.
- Complete the actual-result column in the testing table.
- Insert genuine screenshots from the running project.
- Remove all temporary placeholders before printing or submitting the report.
- Keep all passwords, API keys and private credentials out of the report.